

WEEK 8 SUMMARY

Advanced Queries, Redis Caching, and Performance Tuning in ASP.NET Core

Program: Backend Development Internship (.NET)
Week: Week 8 of 10 | Phase 3: Applied Project Work, Sprint 3
Theme: Advanced EF Core Queries, Redis Caching, and Measured Performance Tuning
Duration: 40 hours across five training days

Executive Summary

Week 8 focused on making the capstone API measurably faster through better Entity Framework Core queries, Redis caching, and targeted database indexing. The central outcome was a performance-oriented API that avoids unnecessary database round-trips, serves suitable high-read data from a distributed cache, and uses indexes that match real filtering and sorting patterns.

The sprint treated performance as an evidence-based engineering task rather than an assumption. Interns established realistic baselines, diagnosed the N+1 query problem through SQL logging, selected eager loading, projection, or split queries according to each endpoint, tested cache invalidation on every related write path, and compared query counts, response times, and execution plans before and after each optimization.

Learning Objectives Achieved

- Diagnosed N+1 query behavior using EF Core SQL logging and realistic seeded data.
- Reduced unnecessary database round-trips through eager loading, projection, and split queries.
- Integrated Redis through IDistributedCache and applied the cache-aside pattern to suitable data.
- Implemented write-side cache invalidation so cached results remain consistent after changes.
- Added targeted and composite database indexes based on actual query patterns.
- Demonstrated performance improvements with before-and-after query counts, timings, and execution plans.

Five-Day Curriculum and Practical Work

Day	Focus	Key Outcome
1	N+1 Diagnosis	Sprint planning, EF Core query logging, realistic seed data, and a measured baseline.
2	Query Optimization	Eager loading, projection, split queries, and verified query-count reduction.

Day	Focus	Key Outcome
3	Redis Caching	Cache-aside integration, expiration policy, write-side invalidation, and hit/miss timing.
4	Indexing & Profiling	Targeted composite indexes, execution-plan analysis, and mentor code review.
5	Benchmark Demo	Sprint review, before/after evidence, iteration backlog, and retrospective.

Day 1: Sprint Planning and N+1 Diagnosis

Sprint 3 began with a measurable goal tied to the capstone's real endpoints. Instead of defining success as simply making the API faster, the backlog used concrete targets such as reducing the query count of an order-history endpoint from more than twenty database calls to one or two. The plan also carried forward one improvement from the Sprint 2 retrospective.

The N+1 problem was examined as one query for a parent list followed by one additional query for each returned item. EF Core query logging made the generated SQL visible, while realistic seed data exposed behavior that would remain hidden with only a few test records. List endpoints that accessed navigation properties inside loops were treated as the strongest candidates for investigation.

Hands-on result: Enabled EF Core query logging, seeded at least 50 realistic records, measured the query counts of key list endpoints, and documented a genuine N+1 issue as a Sprint 3 backlog task.

Day 2: Query Optimization with Eager Loading and Projection

Eager loading with Include and ThenInclude was used when an endpoint genuinely needed related entities. For list and summary responses that required only selected fields, projection with Select produced leaner SQL and avoided loading complete entity graphs. When multiple collection navigations risked a cartesian-product explosion, AsSplitQuery provided a controlled tradeoff between result size and additional round-trips.

Each change was verified by rerunning the same endpoint under the same data conditions. A successful fix showed the query count collapse from N+1 to a small, stable number and preserved the endpoint's functional response. The choice among Include, projection, and split queries was based on the shape of the required data rather than a single rule applied everywhere.

- Use Include for detail views that require complete related entities.
- Prefer projection for list or summary endpoints that need only a limited set of fields.
- Use split queries when multiple included collections would create excessive duplicated rows.

Day 3: Redis Caching and Cache Invalidation

Redis was introduced for data that is read frequently and changes infrequently, such as a product catalog or category tree. ASP.NET Core's IDistributedCache abstraction kept cache access behind a

framework interface, while the cache-aside pattern checked Redis first, loaded from the database on a miss, stored the serialized result with an expiration, and then returned it to the caller.

Cache correctness received the same attention as cache speed. Every create, update, or delete operation that affected a cached result removed or refreshed the relevant cache entry. This prevented stale responses from surviving until expiration and avoided using caching on rapidly changing data where invalidation cost would outweigh the benefit.

Hands-on result: Configured Redis, cached one high-read endpoint, verified immediate freshness after related writes, and recorded the response-time difference between a cache miss and a cache hit.

Day 4: Database Indexing and Performance Profiling

Indexing decisions targeted columns used repeatedly in WHERE clauses, JOIN conditions, and ORDER BY operations. A composite index was added where a common query filtered and sorted on multiple columns together, such as CustomerId and OrderDate. Indexes were not added indiscriminately because each additional index increases storage and write overhead.

EF Core migrations captured the schema changes, and database execution plans showed whether the optimizer moved from an expensive scan to an index seek. The mid-sprint pull request included the reasoning for each query, caching, and indexing choice together with before-and-after evidence for mentor review.

Review standard: An optimization counted as complete only when measurements or execution plans showed a real improvement and existing endpoint behavior remained correct.

Day 5: Sprint Review, Benchmark Demo, and Retrospective

The Sprint Review presented concrete results: query counts before and after eager loading or projection, cache miss and hit timings, and execution plans before and after indexing. Each backlog item was checked against its stated target, the cache invalidation path was demonstrated, and completed work was linked to a reviewed and merged pull request.

Unfinished performance opportunities moved into the Sprint 4 backlog rather than being treated as complete. The retrospective recorded what worked, what should improve, and one specific action for the next sprint, such as adding an automated regression test that detects a reintroduced N+1 query.

Hands-on result: Delivered the benchmark demo, documented remaining opportunities, completed the Sprint 3 retrospective, and assembled the mentor check-in summary with measurements, caching strategy, and pull-request evidence.

Required Deliverables

- A documented N+1 diagnosis with before-and-after query counts and an appropriate EF Core fix.
- Redis cache-aside behavior on at least one high-read endpoint with tested write-side invalidation.
- At least one targeted composite index supported by before-and-after profiling evidence.
- A reviewed and merged pull request containing the performance changes and measurements.

- A Sprint 3 retrospective with one concrete action for Sprint 4 testing, documentation, or deployment.

Performance Verification Checklist

1. Run the same endpoints against realistic seeded data and record baseline query counts and response times.
2. Confirm that optimized endpoints return the same functional results while using the intended query strategy.
3. Test Redis cache misses, cache hits, expiration, and every related create, update, and delete invalidation path.
4. Compare execution plans or timings before and after each index and verify that write overhead remains justified.
5. Attach the evidence to the pull request, demo it during the Sprint Review, and convert suitable checks into regression tests.